Access Controls Policy

Application: SpendWise — Personal Spending Analysis Dashboard
Operator: Ryan Snyder (sole developer and operator)
Last reviewed: 2026-05-16

1. Purpose and scope

This policy describes the controls in place to limit access to production assets and sensitive consumer data within the SpendWise application. It is a companion to the application's Information Security Policy and is intended to be read alongside that document. The scope covers:

- End-user access to data inside the application (user-to-data authorization).
- Operator access to production infrastructure (Vercel hosting, Postgres database, GitHub source repository, Plaid developer dashboard, deployment platform secrets).
- Service-to-service access from the application to third-party APIs (Plaid).

This is a non-commercial, sole-operator application built for friends and family. There are no employees or contractors. Controls are appropriate to that scale.

2. Roles

Two roles exist within the system:

- **End user** — a friend or family member with an account. Has read/write access only to their own data via the application's UI and API.
- **Operator** — the sole developer (Ryan Snyder). Has administrative access to the underlying infrastructure (hosting account, database, deployment, secret store) but **not** to per-user data through any in-application admin interface. The application has no admin role, no impersonation feature, and no cross-user data access path. Operator access to user data, when needed for incident response or support, is only possible via direct database query at the infrastructure layer.

There is no third role (employee, contractor, vendor staff).

3. End-user authentication

- **Identity** is established by email + password during registration. Email addresses are unique.
- **Password storage** uses bcrypt with cost factor 10. Plaintext passwords are never stored or logged. Password verification uses constant-time comparison via the `bcryptjs` library.
- **Session management** is JWT-based (NextAuth v4). JWTs are signed with a per-deployment secret (`NEXTAUTH_SECRET`) held in environment variables. Sessions are HTTP-only cookies and are validated server-side on every authenticated request.
- **Multi-factor authentication (MFA)** is available to all end users and recommended:
- **Method:** Time-based One-Time Password (TOTP, RFC 6238) compatible with Google Authenticator, Authy, 1Password, and equivalent apps.

- **Storage:** the TOTP shared secret is encrypted with AES-256-GCM before being written to the database. The encryption key is held in a separate environment variable (`MFA_ENCRYPTION_KEY`), not in the database.
- **Backup codes:** ten one-time backup codes are generated at enrollment, displayed once, and stored bcrypt-hashed. Used codes are marked consumed atomically to prevent reuse.
- **Login flow:** when MFA is enabled for an account, sign-in requires the password plus a valid TOTP code (or unused backup code). MFA cannot be bypassed by any in-application path. Operator-side MFA reset would require direct database modification, with the account holder notified.

4. End-user authorization

- The application implements per-user data isolation at every authenticated entry point:
- Every server-rendered page reads the authenticated session and either redirects unauthenticated users or scopes queries to `session.user.id`.
- Every API route checks `session.user.id` before any database operation, and every query filter includes that user ID.
- There is no admin role, no super-user role, no "view as another user" capability, and no cross-user data access path. The principle of least privilege is enforced structurally: a query without a `userId` filter would return no data, and an attempt to access another user's resource by ID would return 401/404.
- Cascading foreign-key relationships in the database (`onDelete: Cascade`) ensure that deleting a user removes all of their files, transactions, Plaid connections, MFA data, and preferences atomically.

5. Operator (administrative) access

The operator's administrative access is limited to the underlying infrastructure layer, not in-application administration. Each administrative surface is protected with multi-factor authentication on the operator's account:

Surface	What it grants	MFA enabled
Vercel hosting account	Deployment, environment variables, project settings	Yes (TOTP + passkeys)
Vercel Postgres database	Direct SQL access to all rows	Yes (covered by Vercel account MFA)
GitHub source repository	Source code, branch protection, GitHub Actions	Yes
Plaid developer dashboard	API keys, item management, support tools	Yes
Operator email account	Password resets for the above	Yes

Operator access is used exclusively for application development, deployment, support, and incident response. No routine business process involves the operator reading per-user data.

6. Service-to-service authentication

The application authenticates to Plaid using server-to-server credentials:

- **Application-level identity** with Plaid uses `PLAID_CLIENT_ID` and `PLAID_SECRET`, transmitted as HTTP headers over TLS 1.2+. These secrets are held in environment variables, never in source control.
- **Per-user authorization** with Plaid uses an `access_token` issued by Plaid via the Plaid Link OAuth flow that the end user initiates. The application stores each access token encrypted with AES-256-GCM (`PLAID_TOKEN_ENCRYPTION_KEY`) so that a database dump alone does not expose usable tokens.
- **TLS** is used for all server-to-Plaid traffic via the official Plaid Node SDK. The application does not accept Plaid responses over plaintext channels.

No other service-to-service authentication exists in the system.

7. Access provisioning and de-provisioning

- **End-user provisioning** is self-service: a user registers via the application's `/register` page, providing an email and password. No operator action is required.
- **End-user de-provisioning** is user-initiated: a user can request account deletion, which triggers a cascading delete of all associated data. There is no separate "deactivation" state; deletion is final (subject to the backup-retention window noted in §10).
- **Bank-connection de-provisioning** is user-initiated and immediate: a user can disconnect any connected bank from the Files page, which calls Plaid's `itemRemove` to invalidate the access token on Plaid's side and deletes the local item record (along with associated transactions).
- **Operator provisioning** is a no-op because there are no other operators. If a future operator joined, their access would be provisioned by adding them as a member to each of the surfaces in §5, each of which requires MFA enrollment by the new member before access is granted.
- **Operator de-provisioning** (for example, if the sole operator changed) would proceed by rotating every secret listed in §6 of the Information Security Policy, revoking the prior operator's membership on each surface in §5, and updating the contact email in the privacy policy.

8. Privileged access management

The operator's account on each surface in §5 is, by virtue of being the sole account, privileged. The mitigations against single-operator risk are:

- MFA on every administrative surface (see §5).
- All secrets held in env-var stores rather than configuration files, so a stolen laptop or git repo does not expose production credentials.
- No long-lived ambient credentials are kept on the operator's local machine; deployment uses the Vercel CLI's session-scoped token.

9. Logging and monitoring

- The application logs authentication failures, MFA failures, and API errors (without including the values of secrets, passwords, MFA codes, or full transaction descriptions).
- Plaid records all API calls made with the application's client_id; the operator can review them in the Plaid dashboard.
- Vercel records all deployments, environment-variable changes, and inbound HTTP requests; these are reviewable in the Vercel dashboard.
- For a system of this size, monitoring is by routine inspection during development and on user-reported incidents, rather than an always-on SIEM. No alerts are routed to external paging systems.

10. Periodic access reviews

- Because there is only one operator, no employee/contractor access list exists to review.
- The operator periodically inspects:
- The set of active end-user accounts (to identify unexpected accounts).
- The set of active Plaid items (to identify unexpected connections).
- Vercel team membership and GitHub repo collaborators (expected to remain a single member).
- Review frequency: at minimum annually, and ad-hoc following any incident or material change.

11. Incident response for access events

If the operator becomes aware of a suspected unauthorized access event (for example, a leaked secret, an unexpected new user account, a Plaid-side compromise notification, or a sign-in from an unusual location reported by a friend or family member), the response procedure is:

1. Rotate the relevant secret(s): `NEXTAUTH_SECRET`, `MFA_ENCRYPTION_KEY`, `PLAID_TOKEN_ENCRYPTION_KEY`, `PLAID_SECRET`, database password.
2. For Plaid-specific incidents, call `itemRemove` on the affected item(s) to invalidate access tokens with Plaid.
3. For an end-user account compromise, reset the password and (if MFA was bypassed) require MFA re-enrollment.
4. Notify the affected end user directly.
5. If user data appears to have been accessed by an unauthorized party, notify Plaid via the appropriate support channel.

12. Policy review

This policy is reviewed when material changes are made to the application's authentication, authorization, or infrastructure model, and at least annually.